

TASK 8 – Implement and Test the Algorithms

1. Introduction

The main objective of this task is to implement and experimentally test three searching algorithms: Linear Search, Recursive Binary Search, and Hash Table Search. These algorithms are used to locate a student's registration number in a database. Their performance is evaluated by increasing the number of student records from 1,000 to 10,000, 100,000, and 1,000,000.

The experimental testing helps verify whether the actual behaviour of each algorithm agrees with its theoretical time complexity.

2. Algorithms Selected for Implementation

A. Linear Search

Linear Search checks each student record sequentially, starting from the first record, until the required registration number is found or the end of the database is reached.

- **Design approach:** Brute force.
- **Preprocessing:** Not required.
- **Best-case time complexity:** ($O(1)$).
- **Average and worst-case time complexity:** ($O(n)$).

Linear Search is simple to implement and works with both sorted and unsorted records. However, its performance decreases as the number of records increases.

B. Recursive Binary Search

Recursive Binary Search locates a registration number by comparing it with the middle record of a sorted database. If the number does not match, the algorithm searches only the appropriate half of the remaining records. This process continues recursively until the number is found or the search range becomes empty.

- **Design approach:** Divide and conquer.

- **Preprocessing:** Records must be sorted before searching if they are not already sorted.
- **Best-case time complexity:** ($O(1)$).
- **Average and worst-case time complexity:** ($O(\log n)$).

Binary Search is efficient for large, sorted databases because it eliminates approximately half of the remaining search range during each step.

C. Hash Table Search

Hash Table Search uses a hash function to convert a registration number into an index in a hash table. The algorithm uses this index to locate the required record. If two registration numbers map to the same position, a collision-resolution technique, such as linear probing, is used.

- **Design approach:** Hashing.
- **Preprocessing:** A hash table must be constructed from the student records.
- **Average expected time complexity:** ($O(1)$).
- **Worst-case time complexity:** ($O(n)$).

Hash Table Search provides fast lookups when the hash function distributes registration numbers effectively and collisions are limited. It requires additional memory for the hash table.

3. Dataset Selection

The algorithms are tested using four different dataset sizes to observe how their performance changes as the database grows.

Test Number of student records

- 1 1,000
- 2 10,000
- 3 100,000

4 1,000,000

The same registration numbers are used for all three algorithms at each dataset size. The records are sorted for Binary Search, and the Hash Table is constructed using the same set of registration numbers.

4. Test Cases

The following test cases are used to verify the correctness of each algorithm:

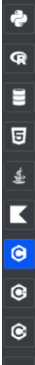
1. **First record:** Search for the first registration number in the database.
2. **Middle record:** Search for a registration number near the middle of the database.
3. **Last record:** Search for the last registration number.
4. **Missing record above the range:** Search for a registration number greater than the largest stored number.
5. **Missing record below the range:** Search for a registration number smaller than the smallest stored number.

Each algorithm should correctly identify whether the registration number exists. A successful search should return a valid position or indicate that the record was found. An unsuccessful search should indicate that the record does not exist.

5. Experimental Testing Procedure

The experiment is conducted using the following steps:

1. Generate student registration numbers for each of the four dataset sizes



main.c

Run ▶

Output

```
40 {  
41     printf("%d ", records[i]);  
42 }  
43  
44 printf("\n\nSearching for Registration Number: %d\n", key);  
45  
46 // Linear Search  
47 if (linearSearch(records, n, key) != -1)  
48     printf("Linear Search: Record Found\n");  
49 else  
50     printf("Linear Search: Record Not Found\n");  
51  
52 // Binary Search  
53 if (binarySearch(records, 0, n - 1, key) != -1)  
54     printf("Binary Search: Record Found\n");  
55 else  
56     printf("Binary Search: Record Not Found\n");  
57  
58 return 0;
```

Student Registration Numbers:
101 105 110 115 120

Searching for Registration Number: 115
Linear Search: Record Found
Binary Search: Record Found